

07-agents

19. Agent Architecture

Helios is a **multi-agent plan-execute-critique system** built on the Claude Agent SDK. Seven agents collaborate under a deterministic control plane: an Orchestrator plans, five worker agents execute one stage each of the diagnosis pipeline, and an adversarial Critic attempts to refute every finding before it ships. The LLM is never the system of record — it is a *composer* that emits tool calls. All SQL flows through `semantic-mcp`; all math flows through `stats-mcp`. This satisfies the **grounding-over-generation** principle: zero hallucinated columns/metrics (100% governed SQL) and 100% of shipped findings carrying a significance test and a dollar revenue-at-risk.

19.1 The seven agents

Agent	Model	Responsibility	Reads	Writes	MCP tools permitted
Orchestrator	Opus	Plan the run, sequence stages, manage budget, route to Critic, finalize	run config, memory	run plan, final brief pointer	<code>warehouse-mcp.list_tables/describe_table</code> , <code>report-mcp.recall_prior</code> , <code>semantic-mcp.list_dimensions</code>
Monitor	Sonnet	Detect which metrics/segments moved abnormally	governed metric series	anomaly list (metric, dim, t0→t1, score)	<code>semantic-mcp.get_metric/build_query</code> , <code>warehouse-mcp.dry_run/run_query</code> , <code>stats-mcp.detect_anomaly/forecast</code>
Decompose	Sonnet	Split each anomaly into mix-shift vs rate-change vs interaction	anomaly list	decomposition table per anomaly	<code>semantic-mcp.build_query</code> , <code>warehouse-mcp.run_query</code> , <code>stats-mcp.decompose_change</code>
Diagnose	Opus	Run hypothesis-tree RCA over the dimensional space; verify each branch with SQL	decompositions	ranked root-cause candidates w/ evidence	<code>semantic-mcp.get_metric/build_query/list_dimensions</code> , <code>warehouse-mcp.dry_run/run_query</code> , <code>stats-mcp.significance_test/decompose_change/cohort_retention</code>
Prescribe	Sonnet	Convert confirmed root causes into a prioritized, powered experiment backlog	confirmed findings	experiment test cards (n, runtime, MDE)	<code>experiment-mcp.power_analysis/runtime_estimate/design_experiment</code> , <code>semantic-mcp.get_metric</code>
Critic	Opus	Adversarially refute every finding (confound, sample, seasonality, data quality)	findings + all evidence	verdict per finding (PASS/DOWNGRADE/DROP) + reasons	<code>semantic-mcp.build_query</code> , <code>warehouse-mcp.run_query</code> , <code>stats-mcp.significance_test/decompose_change</code> , <code>report-mcp.recall_prior</code>
Narrator	Sonnet	Compose the executive Decision Brief from surviving findings	PASS findings + backlog	rendered brief, persisted diagnosis	<code>report-mcp.render_brief/save_diagnosis/export</code> , <code>semantic-mcp.get_metric</code>

Model choice rationale

Opus is reserved for the three roles requiring deep multi-step reasoning over an open hypothesis space and high cost-of-error: **Orchestrator** (global planning, budget allocation, branch routing), **Diagnose** (combinatorial hypothesis-tree search with self-directed SQL verification), and **Critic** (adversarial reasoning that must out-think the Diagnose agent to find confounds). **Sonnet** handles the high-volume, more mechanical stages — **Monitor, Decompose, Prescribe, Narrator** — where the action space is bounded (which metric to series-test, which decomposition to run, which power-analysis to call, which template to fill). This split keeps cost per run inside the fixed BigQuery + token budget while preserving accuracy on the reasoning-critical paths.

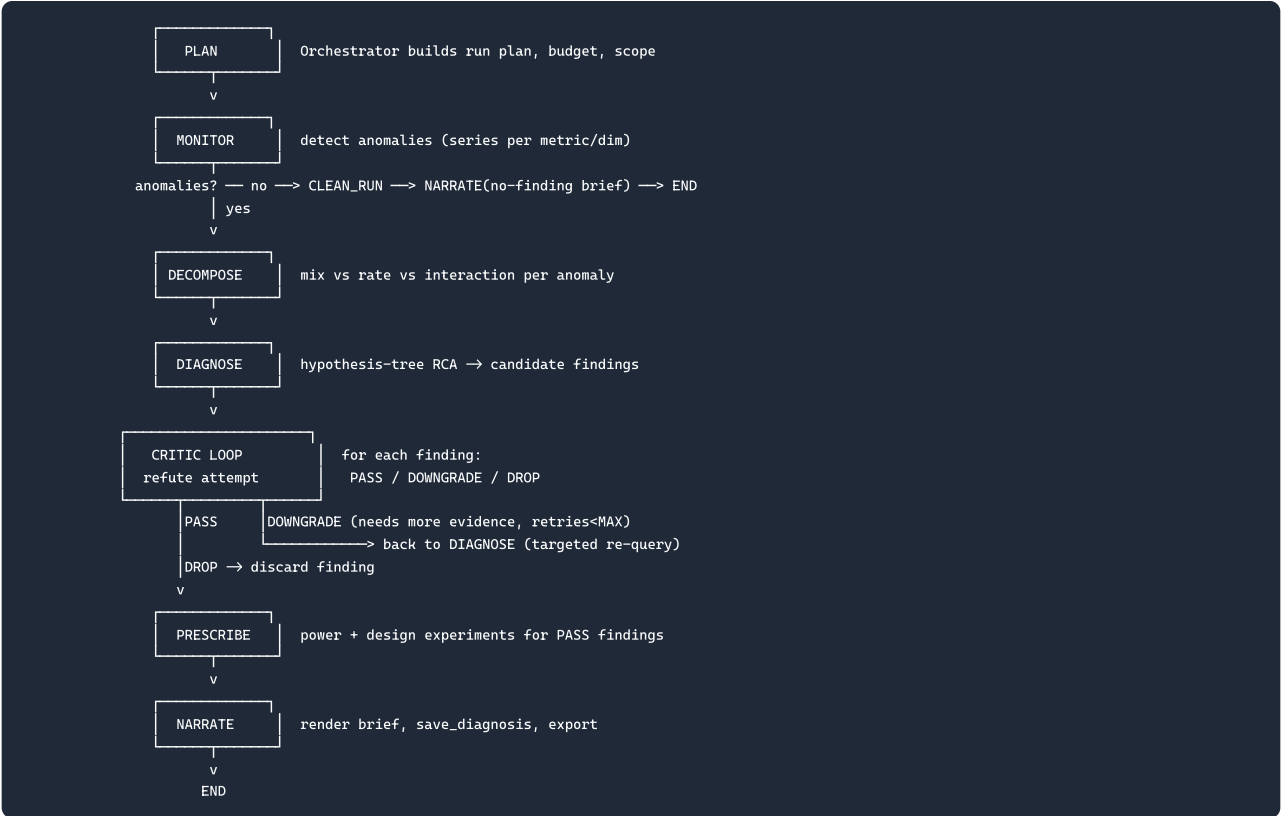
19.2 Grounding rules (non-negotiable)

```
RULE G1 The LLM NEVER emits raw SQL. To get data it MUST call
semantic-mcp.build_query(metric, dims, filters, window) or
semantic-mcp.get_metric(name). build_query returns VALIDATED SQL
(governed metric defs + known dimensions only).
RULE G2 The LLM NEVER computes a statistic in prose. Anomaly scores,
decompositions, significance, forecasts, power → stats-mcp /
experiment-mcp ONLY. Numbers in the brief are tool outputs verbatim.
RULE G3 Every query is dry-run (warehouse-mcp.dry_run) BEFORE run_query.
If bytes_scanned would exceed the per-run byte budget the call is
rejected and the agent must narrow window/dims.
RULE G4 Reconcile: aggregate metrics are checked against
warehouse-mcp.reconcile(metric, grain) canonical totals; a >0.5%
drift fails the finding.
RULE G5 Use ONLY canonical metric/dimension names. semantic-mcp rejects
unknown names (anti-hallucination). An unknown name is a hard error,
not a fallback to free SQL.
```

These rules are enforced twice: structurally (the agents are given *only* the MCP tools in their row above — they physically cannot call `run_query` with arbitrary SQL because they never hold a raw-SQL tool) and behaviorally (each system prompt restates the rules).

19.3 Control flow — the state machine

The Orchestrator drives a finite state machine. Worker output never ships directly; it must pass the Critic loop.



Critic verdicts. **PASS** — finding survives all refutation attempts; proceeds to Prescribe. **DOWNGRADE** — finding is plausible but evidence is insufficient (e.g. sample too small, decomposition not isolated to one dimension); returns to Diagnose for a targeted re-query, bounded by `MAX_REFUTE_ROUNDS = 2`. After the cap, a still-downgraded finding is demoted to a "watchlist" note in the brief rather than a confident finding.

DROP — finding is refuted (confound found, fails reconciliation, fully explained by known seasonality from memory); it is discarded and the cause is added to the run's local suppression so siblings don't re-raise it.

19.4 Hypothesis-tree RCA (Diagnose agent)

Diagnose treats root-cause analysis as a **best-first search over the dimensional space**. The root is the top-level moved metric (e.g. `session_conversion_rate` fell). Each tree node is `(metric, dimension-slice, decomposition verdict)`. Children expand the node along the next canonical dimension, ordered by the **rate-effect magnitude** returned by `stats-mcp.decompose_change` — the agent always drills into the slice contributing the largest *rate* change (a genuine behavior change) before chasing *mix* effects (composition change, which is often a Simpson's-paradox artifact, not a cause).

```
ROOT: session_conversion_rate -2.1pp WoW (revenue-at-risk computed at leaf)
├── split by device_category
│   ├── mobile rate_effect=-1.6pp share+4pp ← expand (largest rate effect)
│   │   └── split by channel_group
│   │       ├── Paid Search rate_effect=-1.3pp ← expand
│   │       │   └── split by landing_page → /sale page rate_effect=-1.1pp [LEAF candidate]
│   │       └── Organic Search rate_effect=-0.1pp (prune: below threshold)
│   └── desktop rate_effect=-0.2pp (prune)
└── mix term across device = +0.3pp (note: NOT a cause; composition shift)
```

Pruning rules: a branch is pruned when its rate-effect is below `MIN_RATE_EFFECT` (a configured pp threshold) OR `significance_test` p-value > 0.05 OR the slice's session count is below the minimum sample for the metric. **Leaf promotion:** a node becomes a candidate finding when it is statistically significant, isolates a single dimension's rate change, survives reconciliation (G4), and carries a non-trivial dollar impact. Search is breadth-bounded (`MAX_BRANCHING = 4` slices per node) and depth-bounded (`MAX_DEPTH = 3` dimensions) to keep query cost and time-to-diagnosis (<5 min/run) in budget. Each leaf is annotated with **revenue-at-risk** = (counterfactual `session_conversion_rate` at t0 – observed at t1) × affected `sessions` × `aov`, computed entirely from governed metrics.

19.5 Context management, retries, error handling

- **Context windowing.** Each worker agent receives a *compacted* context: the run plan, the canonical FOUNDATION metric/dimension catalog, the upstream stage's structured output (JSON, not prose), and any priors surfaced by `recall_prior`. Raw query result sets are summarized to aggregates before entering the LLM context; the LLM never sees row-level dumps. Inter-agent hand-off is a typed JSON envelope (`finding_id`, `metric`, `dims`, `t0`, `t1`, `decomposition`, `significance`, `dollar_impact`, `evidence_query_ids`), keeping token usage flat as the dimensional search widens.
- **Retries.** Transient `warehouse-mcp` / BigQuery errors retry with exponential backoff (3 attempts). A `dry_run` budget rejection (G3) is *not* retrieved blindly — the agent must narrow scope (shorter window or fewer dims) before re-issuing. A `semantic-mcp` unknown-name error (G5) is a hard stop for that branch; the agent re-plans the query against `list_dimensions()`.
- **Failure isolation.** A single failed branch in Diagnose is logged and pruned, not fatal to the run. If Monitor/Decompose fail entirely, the Orchestrator aborts and writes an audit record (no partial brief ships). Every tool call, its arguments hash, bytes scanned, and verdict are appended to the run audit trail (see 22.5) so a run is fully reconstructable.

19.6 End-to-end autonomous run — sequence diagram

```
Scheduler(cron/Cloud Scheduler) —fire→ Orchestrator
Orchestrator → report-mcp.recall_prior(metric, segment) # load priors, suppression
Orchestrator → warehouse-mcp.list_tables / semantic-mcp.list_dimensions # scope
Orchestrator → Monitor: "diagnose window t0..t1, budget B"
Monitor → semantic-mcp.build_query(session_conversion_rate, [day], window)
Monitor → warehouse-mcp.dry_run → run_query
Monitor → stats-mcp.detect_anomaly(series) → [anomaly: conv_rate -2.1pp]
Monitor → Orchestrator: anomaly list
Orchestrator → Decompose: anomaly list
Decompose → stats-mcp.decompose_change(conv_rate, device_category, t0, t1)
Decompose → Orchestrator: {mix:+0.3pp, rate:-1.8pp, interaction:-0.6pp}
Orchestrator → Diagnose: decompositions
Diagnose → (loop) semantic-mcp.build_query + warehouse-mcp.run_query
Diagnose → stats-mcp.decompose_change / significance_test # walk hypothesis tree
Diagnose → Orchestrator: candidate finding F1 (/sale Paid Search mobile, p=0.01, $42k risk)
Orchestrator → Critic: F1 + evidence
Critic → report-mcp.recall_prior # known seasonality? prior suppression?
Critic → stats-mcp.significance_test (re-run on holdout slice)
Critic → warehouse-mcp.run_query (confound probe: did traffic mix flip?)
Critic → Orchestrator: VERDICT=PASS (no confound; not seasonal; reconciles)
Orchestrator → Prescribe: F1
Prescribe → experiment-mcp.power_analysis(baseline, mde) → n
Prescribe → experiment-mcp.runtime_estimate(n, traffic) → 9 days
Prescribe → experiment-mcp.design_experiment(hypothesis, metric) → test card
```

```

Prescribe → Orchestrator: backlog[F1 → Exp-001]
Orchestrator → Narrator: PASS findings + backlog
Narrator → report-mcp.render_brief(findings)
Narrator → report-mcp.save_diagnosis(...) # writes memory (sec 22)
Narrator → report-mcp.export(format=pdf/slack)
Narrator → Orchestrator: brief URL
Orchestrator → Scheduler: run complete, audit row written

```

A clean run (no anomaly) short-circuits after Monitor and Narrator emits a "no material change" brief, still writing a run-state audit row so absence-of-finding is itself recorded.

22. Memory Architecture

Memory is what turns Helios from a stateless analyst into one that **learns across runs**: it stops re-flagging acknowledged causes, recognizes known seasonality and launches, and closes the loop by checking whether prescribed experiments actually moved the metric. Memory is split across **BigQuery tables** (the durable system of record, queryable by the same `warehouse-mcp`) and a **vector store** (embeddings of past findings for similarity recall). All memory I/O for agents goes through `report-mcp` (`save_diagnosis`, `recall_prior`) so the LLM never writes memory directly.

22.1 Diagnosis history

Every shipped (and DROPPED, for audit) finding is persisted with an embedding of its natural-language summary so future runs can find similar past diagnoses.

```

-- dataset: helios_memory
CREATE TABLE IF NOT EXISTS helios_memory.diagnosis_history (
  finding_id      STRING NOT NULL,      -- uuid
  run_id          STRING NOT NULL,
  created_at      TIMESTAMP NOT NULL,
  metric          STRING NOT NULL,      -- canonical, e.g. session_conversion_rate
  dimension_slice STRING,               -- e.g. device_category=mobile|channel_group=Paid Search
  t0              DATE, t1 DATE,
  direction       STRING,               -- up|down
  magnitude        FLOAT64,             -- delta in metric units (pp or %)
  mix_effect       FLOAT64,
  rate_effect      FLOAT64,
  interaction_effect FLOAT64,
  p_value          FLOAT64,
  dollar_impact    FLOAT64,             -- revenue-at-risk
  critic_verdict   STRING,              -- PASS|DOWNGRADE|DROP
  root_cause_label STRING,              -- canonical cause taxonomy id
  summary_text     STRING,              -- NL one-liner used for embedding
  embedding        ARRAY<FLOAT64>      -- text-embedding vector (also mirrored to vector store)
)
PARTITION BY DATE(created_at)
CLUSTER BY metric, root_cause_label;

```

Write path: Narrator calls `report-mcp.save_diagnosis(finding)` → inserts the row and upserts (`finding_id`, `embedding`, `metric`, `root_cause_label`) into the vector store. **Retrieval path:** `recall_prior(metric, segment)` runs a hybrid query — exact filter on `metric` + `dimension_slice` in BigQuery, plus a vector ANN search on the embedding of the *current* candidate's summary — and returns the top-k priors with their verdicts and recency. **Decay:** priors are weighted by $\exp(-\text{age_days} / \text{HALF_LIFE})$ (default `HALF_LIFE = 60d`, matching the ~3-month dataset window) so stale findings fade but are never hard-deleted (audit requirement).

22.2 Suppression list

Acknowledged or intentionally-ignored causes that must **not** be re-raised, so the brief doesn't repeat itself week after week.

```

CREATE TABLE IF NOT EXISTS helios_memory.suppression_list (
  suppression_id STRING NOT NULL,
  metric          STRING NOT NULL,
  dimension_slice STRING,
  root_cause_label STRING,
  reason          STRING,              -- 'acknowledged_by_owner' | 'known_business_decision' | 'wontfix'
  created_by      STRING,              -- user email or 'critic_auto'
  created_at      TIMESTAMP,
  expires_at      TIMESTAMP            -- TTL; NULL = permanent
)

```

```

)
CLUSTER BY metric, root_cause_label;

```

Write path: a stakeholder acknowledges a finding (via the brief's action UI) → row inserted with `reason` and an `expires_at` TTL (default 30d so a re-emerging issue eventually re-surfaces); the Critic may also auto-suppress within a run. **Retrieval path:** Diagnose and Critic both consult the suppression list before promoting a leaf — a candidate matching an unexpired (`metric`, `dimension_slice`, `root_cause_label`) is auto-DROP ped with reason "suppressed". **TTL/decay:** rows with `expires_at < now()` are ignored by reads (a scheduled job soft-archives them). This closes the "stop nagging me" loop.

22.3 Business glossary & context

Definitions, known seasonality, and the launch calendar — the priors the Critic uses to refute "this is just seasonal/expected".

```

CREATE TABLE IF NOT EXISTS helios_memory.glossary (
  term          STRING NOT NULL,    -- e.g. 'cart_abandonment_rate'
  definition     STRING,
  canonical_name STRING,            -- maps synonym → canonical metric/dim
  embedding      ARRAY<FLOAT64>
);

CREATE TABLE IF NOT EXISTS helios_memory.seasonality_calendar (
  event_label   STRING,    -- 'Black Friday', 'Christmas dip', 'New Year'
  start_date    DATE, end_date DATE,
  metric        STRING,    -- affected metric, NULL = all
  expected_dir  STRING,    -- up|down
  expected_mag  FLOAT64,   -- typical delta, for confound subtraction
  notes        STRING
);

CREATE TABLE IF NOT EXISTS helios_memory.launch_calendar (
  launch_id    STRING,
  launch_date  DATE,
  description  STRING,    -- 'new /sale landing page', 'checkout redesign'
  affected_dims STRING,   -- e.g. landing_page=/sale
  affected_metric STRING
);

```

Write path: seeded once from the FOUNDATION canonical names (glossary) and the GA4 sample window's known events (the dataset spans 2020-11-01→2021-01-31, so Black Friday 2020 and the December peak/January trough are pre-loaded into `seasonality_calendar`); thereafter updated by analysts. **Retrieval path:** the Critic queries `seasonality_calendar` and `launch_calendar` overlapping `[t0,t1]` for the finding's metric/slice; if the observed change is within `expected_mag` of a calendar entry, the finding is DROP ped or DOWNGRADE d as "explained by known seasonality/launch". The glossary resolves any analyst-typed synonym in the drill-down chat back to a canonical name before it reaches `semantic-mcp` (enforcing G5). **TTL:** glossary/calendar entries are durable (no decay).

22.4 Action tracking

Which prescribed experiments shipped, and whether the fix actually worked — the **did-the-fix-work** loop.

```

CREATE TABLE IF NOT EXISTS helios_memory.action_tracking (
  experiment_id  STRING NOT NULL,
  finding_id     STRING NOT NULL,    -- FK → diagnosis_history
  hypothesis     STRING,
  target_metric  STRING,             -- canonical
  baseline_value FLOAT64,
  mde            FLOAT64,            -- minimum detectable effect from power_analysis
  required_n     INT64,
  est_runtime_days INT64,
  status         STRING,             -- proposed|shipped|running|completed|abandoned
  shipped_at     TIMESTAMP,
  result_lift    FLOAT64,            -- observed effect once completed
  result_p_value FLOAT64,
  outcome        STRING,             -- win|flat|loss|inconclusive
  updated_at     TIMESTAMP
)
CLUSTER BY target_metric, status;

```

Write path: Prescribe inserts a `proposed` row per test card (with `required_n`, `mde`, `est_runtime_days` from `experiment-mcp`); a webhook/manual update from the experimentation platform advances `status` and writes `result_lift` / `outcome` on completion. **Retrieval path:** at the start of each run, the Orchestrator calls `recall_prior` which joins `action_tracking` so the brief reports "Exp-001 for the /sale finding completed: +1.4pp conv, p=0.02, WIN" and so Diagnose down-weights a root cause whose fix already shipped and won. **Loop closure:** an `outcome`

= win validates the prior diagnosis (raises confidence on that `root_cause_label` in future similarity scoring); an `outcome = loss/flat` flags the original finding as a likely false diagnosis, lowering its prior weight — Helios literally learns which of its diagnoses were correct.

22.5 Run state / audit trail

Full reconstructability of every autonomous run — the determinism and verify-then-trust principles depend on it.

```
CREATE TABLE IF NOT EXISTS helios_memory.run_state (  
  run_id          STRING NOT NULL,  
  started_at      TIMESTAMP, ended_at TIMESTAMP,  
  trigger         STRING,      -- 'scheduler' | 'manual'  
  window_t0       DATE, window_t1 DATE,  
  state           STRING,      -- PLAN|MONITOR|...|END|ABORTED  
  bytes_scanned   INT64,      -- vs per-run byte budget  
  total_cost_usd  FLOAT64,  
  n_findings      INT64, n_passed INT64, n_dropped INT64,  
  brief_uri       STRING  
);  
  
CREATE TABLE IF NOT EXISTS helios_memory.audit_log (  
  run_id          STRING, step_seq INT64,  
  agent           STRING,      -- Orchestrator|Monitor|...  
  mcp_tool        STRING,      -- e.g. semantic-mcp.build_query  
  args_hash       STRING,      -- sha256 of normalized args  
  sql_text        STRING,      -- governed SQL actually run (NULL for non-query tools)  
  bytes_scanned   INT64,  
  latency_ms      INT64,  
  verdict         STRING,      -- for Critic steps  
  ts              TIMESTAMP  
)  
PARTITION BY DATE(ts);
```

Write path: the Orchestrator opens a `run_state` row at `PLAN` and updates `state`/budget counters at each transition; every MCP call from any agent appends an `audit_log` row (the control plane wraps all tool invocations). **Retrieval path:** internal — used to enforce the per-run BigQuery byte budget mid-run (if `bytes_scanned` nears the cap the Orchestrator tightens scope), to verify the "0 hallucinated metrics / 100% governed SQL" target by proving every `sql_text` came from `semantic-mcp`, and to power the offline eval harness that grades root-cause accuracy (target $\geq 85\%$ vs $\leq 45\%$ naive baseline). **TTL:** durable; partitioned by day for cheap retention management.

22.6 How memory closes the loop

```
RUN N    diagnose F → save_diagnosis (history + embedding)  
          → prescribe Exp → action_tracking(proposed)  
stakeholder ack → suppression_list (TTL 30d)  
Exp ships+completes → action_tracking(outcome=win/loss)  
RUN N+1 recall_prior(metric, segment):  
  - similar prior F via vector ANN → "seen before, here is the trend"  
  - suppression hit                → auto-DROP, don't re-nag  
  - seasonality/launch overlap      → Critic refutes as expected  
  - action outcome=win              → down-weight already-fixed cause  
  - action outcome=loss             → lower prior confidence (was a bad diagnosis)
```

Each memory type therefore feeds a distinct learning signal: history gives continuity and similarity, suppression prevents repetition, glossary/calendar supplies the confound priors the Critic needs, action tracking validates or invalidates past diagnoses, and the audit trail guarantees every run is governed, budgeted, and reproducible.